

Annexe 1 : Déploiement d'un environnement de développement portable

1. Contexte technique	2
2. Architecture de ma solution	2
3. Preuves matérielles	3
4. Bilan	4

1. Contexte technique

Dès mon arrivée en stage, je devais mettre en place mon environnement pour développer mes scripts d'automatisation. N'ayant pas les droits d'administrateur, je ne pouvais pas faire d'installations libres. En consultant le catalogue d'applications (store interne) de l'entreprise, j'ai constaté que l'éditeur **Spyder** y était disponible, mais qu'une installation de **Python** en standalone (seul) ne l'était pas. Faire une demande de droits à la **DSI** aurait pris un temps incompatible avec les délais de mon projet. J'ai donc dû concevoir une solution technique pour travailler avec les ressources autorisées.

2. Architecture de ma solution

Pour obtenir un environnement fonctionnel et isolé, j'ai procédé ainsi :

- Exploitation de l'interpréteur embarqué : J'ai installé **Spyder** via le store de l'intranet. Pour contourner l'absence de **Python** sur le système, j'ai utilisé l'interpréteur **Python** qui est nativement intégré dans les fichiers d'installation de **Spyder**. J'ai ensuite configuré **Windows** pour que mes fichiers **.py** s'ouvrent et s'exécutent par défaut avec cet exécutable précis.
- Navigateur de test isolé : Le navigateur **Chrome** par défaut de l'entreprise est géré par la **DSI** (mises à jour forcées, règles de sécurité). Pour garantir la stabilité de mes tests avec **Documentum**, j'ai téléchargé le binaire autonome "**Chrome for Testing**" que j'ai placé directement à la racine de mon projet. ainsi que l'extension Documentum, directement intégrée à ce navigateur.
- Configuration du **WebDriver** : Dans mon code **Python**, j'ai explicitement instruit **Selenium** de ne pas chercher le **Chrome** du système, mais de cibler l'exécutable **Chrome for Testing** présent dans mon dossier de travail.

3. Preuves matérielles

```
# %% Utilise le Chrome for Testing local
options.binary_location = os.path.join(base_dir, "chrome_env", "chrome-win64", "chromeft.exe")
service = Service(os.path.join(base_dir, "chrome_env", "chromedriver-win64", "chromedriver.exe"))
driver = webdriver.Chrome(service=service, options=options)
```

Figure 1 : Extrait du script ordonnant au **WebDriver Selenium** d'utiliser l'instance locale de **Chrome** plutôt que le navigateur système.

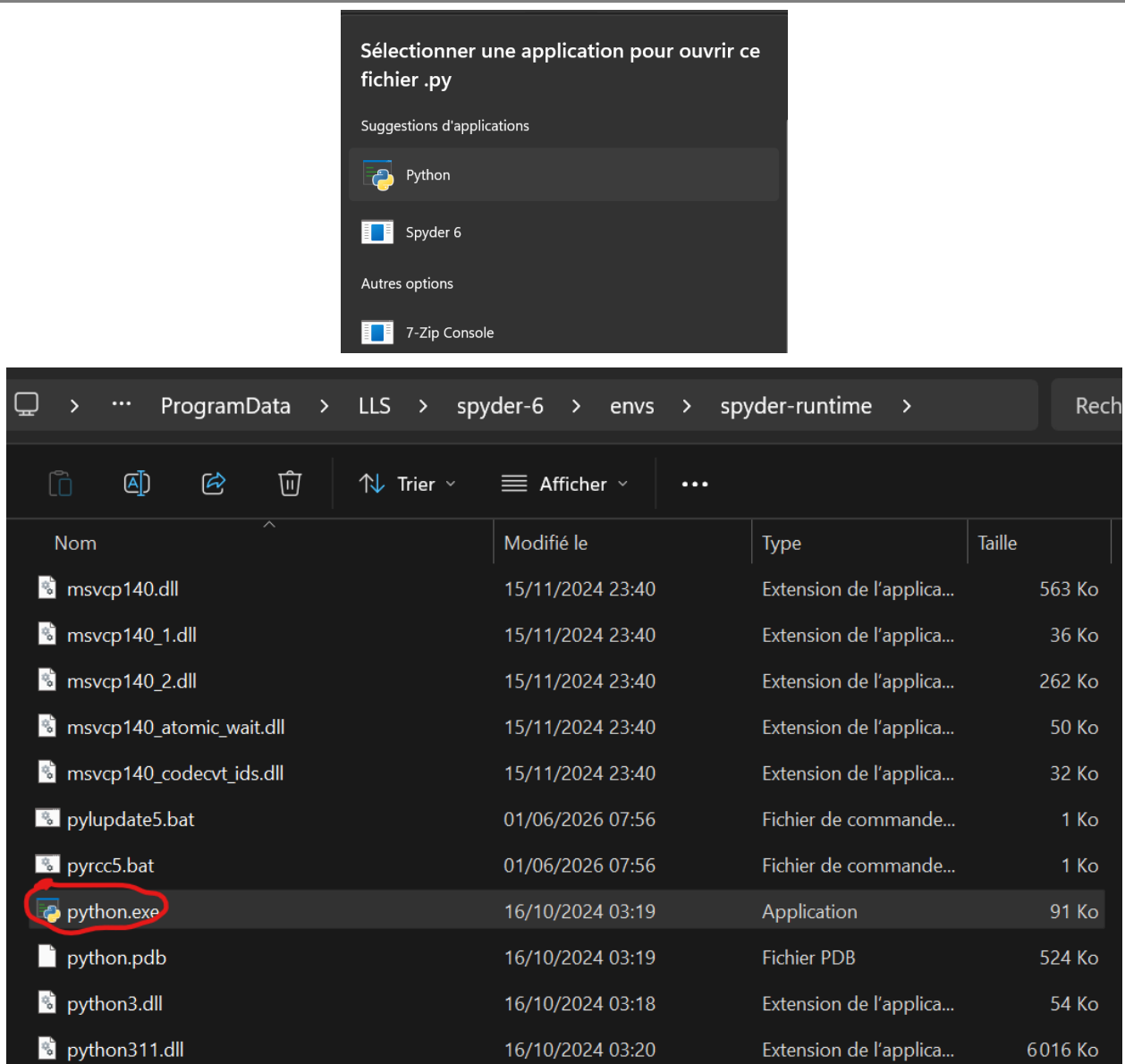


Figure 2 : Configuration du poste de travail forçant l'exécution des scripts via l'interpréteur embarqué de **Spyder**.

4. Bilan

Cette mise en place m'a permis de démarrer mon développement immédiatement tout en respectant les outils validés par l'entreprise. J'ai prouvé ma capacité à recenser les ressources d'un système d'information et à adapter mon code pour créer un environnement d'exécution autonome.